

# **14a — Sortieralgorithmen**

Daten systematisch ordnen — Bubble, Insertion, Quick

---

HTW Berlin – Wirtschaftsingenieurwesen

SoSe 2026

# Agenda

1. Wozu sortieren?
2. Bubble Sort – der einfachste Algorithmus
3. Insertion Sort – “wie Karten in der Hand”
4. Quicksort – Divide-and-Conquer mit Rekursion
5. Pythons `sorted()` und `list.sort()`
6. Wann nehme ich welchen?

**Lernziel:** Sie können drei klassische Sortieralgorithmen erklären, ihre Komplexität nennen und in Python implementieren.

**Wie wir heute arbeiten:** Sortieren *sehen* – jeder Algorithmus hat seine eigene Visualisierung im Terminal.

# Wozu sortieren?

Veggie Soles will eine Bestseller-Liste anzeigen:

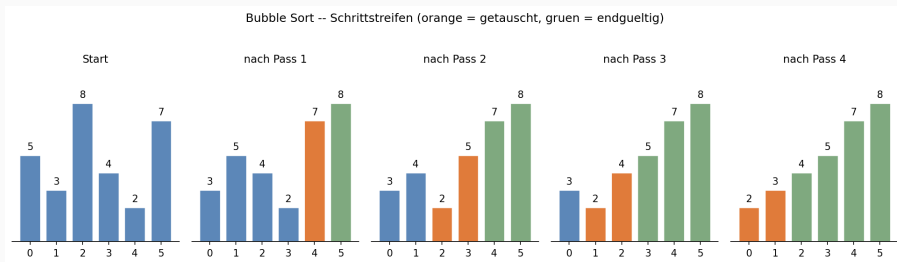
preise = [109.00, 89.95, 135.50]    # *Hemp-High, Eco-Sneaker, Bamb*

- Anzeige nach Preis aufsteigend
- Reports / Top-N-Auswertungen
- **Voraussetzung für Binärsuche** (NB 14b): nur sortierte Listen lassen sich binär durchsuchen
- Klassiker im Vorstellungsgespräch – jeder soll Bubble & Quick erklären können

Heute: drei Algorithmen, **gleiches Ergebnis**, sehr **unterschiedliche Geschwindigkeit**.

# Bubble Sort - die Idee

- Vergleiche zwei **benachbarte** Elemente
- Sind sie in falscher Reihenfolge → tausche sie
- Wiederhole, bis kein Tausch mehr nötig ist
- Größtes Element “blubbert” pro Durchlauf nach hinten



**Abbildung 1:** Bubble Sort - Schrittstreifen

Anschaulich: wie Bücher nach Größe im Regal – Sie tauschen Nachbarn, bis

# Bubble Sort - der Code

```
def bubble_sort(liste):  
    n = len(liste)  
    for i in range(n):  
        for j in range(n - i - 1):  
            if liste[j] > liste[j + 1]:  
                liste[j], liste[j + 1] = liste[j + 1], liste[j]  
    return liste
```

| Wert                  | Komplexität                  |
|-----------------------|------------------------------|
| Zeit (best/avg/worst) | $O(n)$ / $O(n^2)$ / $O(n^2)$ |
| Speicher              | $O(1)$ - in-place            |

► **Live-Demo** - 01\_bubble\_sort\_visuell.py, 02\_bubble\_sort\_balken.py

# Bubble Sort - den Algorithmus *sehen*

Pass 1:

|                         |                              |
|-------------------------|------------------------------|
| [109.00, 89.95, 135.50] | Vergleich (0,1): tausche     |
| [89.95, 109.00, 135.50] | Vergleich (1,2): kein Tausch |

Pass 2:

|                         |                              |
|-------------------------|------------------------------|
| [89.95, 109.00, 135.50] | Vergleich (0,1): kein Tausch |
|-------------------------|------------------------------|

Pro Pass druckt die Demo die Liste mit [ ]-Markern an den getauschten Stellen.

**Visualisierungsstufe 3a:** State-Snapshot pro Schritt – man sieht, wo sich gerade etwas bewegt.

 **Sofort ausprobieren** – Notebook 14a, Abschlussübung 1:  
bubble\_sort\_strings (sortiere Strings nach Länge)

# Insertion Sort - die Idee

- Linker Teil der Liste ist immer sortiert
- Nimm das nächste Element und füge es **an der richtigen Stelle** ein

Start:        [<5>, 3, 8, 4, 2]

Nimm 3:      [<3, 5>, 8, 4, 2]

Nimm 8:      [<3, 5, 8>, 4, 2]

Nimm 4:      [<3, 4, 5, 8>, 2]

Nimm 2:      [<2, 3, 4, 5, 8>]

Anschaulich: Karten in der Hand sortieren – jede neue Karte rutscht an ihren Platz.

# Insertion Sort - der Code

```
def insertion_sort(liste):  
    for i in range(1, len(liste)):  
        schluessel = liste[i]  
        j = i - 1  
        while j >= 0 and liste[j] > schluessel:  
            liste[j + 1] = liste[j]  
            j -= 1  
        liste[j + 1] = schluessel  
    return liste
```

| Wert                  | Komplexität                  |
|-----------------------|------------------------------|
| Zeit (best/avg/worst) | $O(n)$ / $O(n^2)$ / $O(n^2)$ |
| Speicher              | $O(1)$                       |



# Quicksort - Divide and Conquer

- Wähle ein **Pivot**-Element (z. B. erstes)
- Teile in zwei Teile: kleiner-als-Pivot und größer-gleich-Pivot
- Sortiere beide Teile **rekursiv**
- Setze sortierte Teile + Pivot wieder zusammen

```
def quicksort(liste):  
    if len(liste) <= 1:  
        return liste  
    pivot = liste[0]  
    kleiner = [x for x in liste[1:] if x < pivot]  
    groesser = [x for x in liste[1:] if x >= pivot]  
    return quicksort(kleiner) + [pivot] + quicksort(groesser)
```

---

Wert

Komplexität

---

Zeit (best/avg/worst)     $O(n \log n)$  /  $O(n \log n)$  /  $O(n^2)$

## Quicksort - der Rekursionsbaum

```
quicksort([5, 3, 8, 4, 2, 7, 1, 6])  
  pivot=5  kleiner=[3,4,2,1]  groesser=[8,7,6]  
    quicksort([3, 4, 2, 1])  
      pivot=3  kleiner=[2,1]  groesser=[4]  
        quicksort([2, 1])  
          pivot=2  kleiner=[1]  groesser=[]  
            -> [1, 2]  
          -> [1, 2, 3, 4]  
        quicksort([8, 7, 6])  
          -> [6, 7, 8]  
      -> [1, 2, 3, 4, 5, 6, 7, 8]
```

**Visualisierungsstufe 3b:** indentierter Rekursionsbaum - man sieht den ganzen Aufrufbaum auf einmal.

# Vergleich der drei Algorithmen

| Algorithmus     | Best          | Avg           | Worst         | Speicher    | Stabil? |
|-----------------|---------------|---------------|---------------|-------------|---------|
| Bubble Sort     | $O(n)$        | $O(n^2)$      | $O(n^2)$      | $O(1)$      | ja      |
| Insertion Sort  | $O(n)$        | $O(n^2)$      | $O(n^2)$      | $O(1)$      | ja      |
| Quicksort       | $O(n \log n)$ | $O(n \log n)$ | $O(n^2)$      | $O(\log n)$ | nein    |
| Pythons TimSort | $O(n)$        | $O(n \log n)$ | $O(n \log n)$ | $O(n)$      | ja      |

- “Stabil” = gleiche Werte behalten ihre relative Reihenfolge
- Pythons `sorted()` ist in der Praxis fast immer die richtige Wahl

► **Live-Demo** - 05\_sortieren\_vergleich.py

# Pythons eingebaute Sortierung

```
preise = [109.00, 89.95, 135.50]
```

```
# Variante A: neue Liste, Original bleibt
```

```
sortiert = sorted(preise)
```

```
# Variante B: in-place, Original wird verändert
```

```
preise.sort()
```

```
# Mit eigenem Schlüssel: nach Länge sortieren
```

```
namen = ["Eco-Sneaker", "Hemp-High", "Bambus-Boot"]
```

```
sortiert_nach_laenge = sorted(namen, key=len)
```

```
# Absteigend
```

```
sortiert_absteigend = sorted(preise, reverse=True)
```

# Cheat Card

| Algorithmus                                      | Idee in einem Satz                    | Zeit (avg)    |
|--------------------------------------------------|---------------------------------------|---------------|
| Bubble Sort                                      | Nachbarn vergleichen, tauschen        | $O(n^2)$      |
| Insertion Sort                                   | Karte an die richtige Stelle einfügen | $O(n^2)$      |
| Quicksort                                        | Pivot + zwei Hälften, rekursiv        | $O(n \log n)$ |
| <code>sorted()</code> / <code>list.sort()</code> | TimSort – nimm das in der Praxis      | $O(n \log n)$ |

**Faustregel:** Lernen Sie Bubble und Quick zu erklären – nehmen Sie `sorted()`.

## Ausblick: 14b - Suche

Sortierte Listen sind **wertvoll**: man kann sie viel schneller durchsuchen.

```
# Lineare Suche --  $O(n)$ 
```

```
for i, wert in enumerate(liste):  
    if wert == ziel:  
        return i
```

```
# Binäre Suche --  $O(\log n)$  -- braucht sortierte Liste
```

```
links, rechts = 0, len(liste) - 1  
while links <= rechts:  
    mitte = (links + rechts) // 2  
    ...
```

In NB 14b sehen wir: Bei  $n=1\,000\,000$  ist binär ca. 50 000-mal schneller.

# Heute geübt

- ✓ Bubble Sort – Idee, Code, Visualisierung pro Pass
- ✓ Insertion Sort – Idee, Code, Trace mit sortiertem Präfix
- ✓ Quicksort – Idee, Code, Rekursionsbaum
- ✓ Komplexitäten verglichen (best / avg / worst)
- ✓ `sorted()` und `list.sort()` mit `key=` und `reverse=`
- ✓ ASCII-Balken aus 13b auf Sortier-States angewendet

## **Zur Vertiefung im Notebook 14a:**

- Angeleitete Übungen 1.1-3.x: Bubble absteigend, Insertion-Variante, Quicksort-Pivot-Wahl
- Abschlussübungen 1-4: `bubble_sort_strings`, `ist_sortiert`, `top_n_elemente`, `sortiere_woerterbuch_nach_wert`